

# PROJECT-BASED LEARNING

## PHASE-II EVALUATION

**Satellite Health Monitoring and Collision Avoidance**

Team ID: DSCPP-III-2026-T004

**Department of Computer Science & Engineering**  
Graphic Era (Deemed to be University), Dehradun  
Academic Session 2026-27

## Team & Mentor Details

---

### Team Information

**Team Name:** Sentra  
**Team ID:** DSCPP-III-2026-T004  
**Semester:** 3rd  
**Domain:** Object-Oriented / DSA

### Team Members

1. Chhavit – 2510011425
2. Harsh Jain – 2510370400
3. Satyam Kumar – 2510380092
4. Sudhanshu Ojha – 2510011648

### Mentor

**Mentor Name:** Mr. Utkarsh Pant

**Interactions completed:** \_\_\_\_\_

# Phase-I Recap & Feedback Incorporated

---

## Project Idea

A C++ simulation that tracks satellite health and automatically dodges space debris using Data Structures and OOP concepts.

## Feedback from Phase I

- ▶ Working in one big file may cause Git merge conflicts.
- ▶ Program will crash if the user typed letters instead of numbers.
- ▶ Satellite is moving in straight line instead of circular orbit.

## Changes Made for Phase II

- ▶ Divided code into 5 header files to make teamwork easier.
- ▶ Added strict `cin` validation loops to prevent input crashes.
- ▶ Added simulated gravity math so satellites orbit in circles.

# Problem Statement & Finalized Objectives

---

## Problem Statement

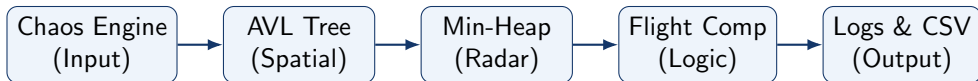
Earth's orbit is filling up with space junk. Satellites need to automatically dodge debris and fix their own internal errors without waiting for slow commands from humans on the ground.

## Phase II Objectives

1. Map the 3D space using an **AVL Tree** to quickly find nearby debris.
2. Use a **Min-Heap** to find the most dangerous, closest threat.
3. Use a **Flight Computer** class to manage fuel and evasive burns.
4. Generate a `telemetry.csv` file to save all coordinates.
5. Use a **Linear Search** to let the user find satellites by name at the end.

# Updated System Architecture / Workflow

---



## Major Components

- ▶ **DSA:** AVL Tree, Min-Heap, Queues
- ▶ **OOP:** Base/Derived Classes, File I/O
- ▶ **Engine:** Gravity math & Random events

## New Backend Features

- ▶ Active fuel consumption.
- ▶ Nearest friendly satellite tracking.
- ▶ Exports data to `telemetry.csv` for Phase III.

# Module Design & Functional Requirements

---

## Major Modules

1. AVL\_Tree.h (Satyam)
2. Queues\_DSA.h (Sudhanshu)
3. Satellites\_OOP.h (Chhavit)
4. Chaos\_Engine.h (Harsh)

## Functional Requirements

- ▶ Fast 3D grid searching.
- ▶ Tracking distance to Earth and other satellites.
- ▶ Safely handling wrong user inputs.
- ▶ Logging all data for the future Web UI.

## Module Dependency

The Chaos\_Engine creates debris and hardware faults. The AVL\_Tree groups debris by location, and passes nearby threats to the MinHeap. The FlightComputer decides if the satellite has enough fuel to dodge the top threat.

# Technology Stack & Development Methodology

---

## Technology Stack

- ▶ Programming Language: **C++**
- ▶ Math & Vectors: `<cmath>`
- ▶ File I/O: `<fstream>` (.txt & .csv)
- ▶ Interface: **Command Line (CLI)**
- ▶ Version Control: **GitHub**

## Development Steps

1. Set up OOP base and derived classes.
2. Implement AVL Tree and Min-Heap.
3. Add circular gravity math.
4. Fix input buffer bugs.
5. Add linear search and CSV data export.

# Implementation of PBL Course Concepts

---

| Course                      | Concepts Actually Implemented                                                                                                    | Project Module                              |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| <b>Data Structures in C</b> | AVL Trees (Spatial grouping), Min-Heaps (Threat sorting), Circular Queues (Battery history), Linear Search (Finding satellites). | AVL_Tree.h,<br>Queues_DSA.h,<br>orbital.cpp |
| <b>OOPs with C++</b>        | Abstract classes, Inheritance, Polymorphism (virtual functions), Operator Overloading (Vector3 Math), File Handling.             | Satellites_OOP.h,<br>FlightComputer.h       |

**Syllabus Note:** We had to rewrite some code to avoid advanced features, ensuring we strictly used raw pointers, manual memory management (delete), and basic loops per our syllabus.

# Development & Implementation Progress

---

| Module                               | Status    | Work Completed                                                     |
|--------------------------------------|-----------|--------------------------------------------------------------------|
| AVL_Tree.h                           | Completed | 3D spatial grid index and simulated gravity calculations.          |
| Queues_DSA.h                         | Completed | Threat sorting Min-Heap & distance math functions.                 |
| Satellites_OOP.h                     | Completed | Fuel consumption, nearest satellite tracking, auto-patching.       |
| Chaos_Engine.h                       | Completed | Solar flares, sensor faults, rogue debris generation.              |
| orbital.cpp<br>(main execution file) | Completed | Input validation, Linear search by name, and CSV Telemetry export. |

**Backend Completion Status: 90-95% Ready for Phase III**

# Working Prototype / Demonstration

---

## Implemented Features

- ▶ **Live Dashboard:** Shows battery, fuel, and software health.
- ▶ **Distance Tracking:** Shows distance to Earth and the nearest friendly satellite.
- ▶ **Collision Dodging:** Uses fuel to automatically avoid debris.
- ▶ **Search Tool:** Find exact coordinates after the simulation ends.

## CLI Dashboard Output

```
CommSat-1 [Communication Satellite]
-> Batt=80% SW=100% Fuel=42kg
-> Earth Dist: 4802.1 units
-> Nearest Sat: ImageSat-2 (154.2 units)

!! COLLISION ALERT !!
ACTION: EVASIVE BURN ACTIVATED
FUEL USED: 2.5 kg
```

# Testing, Results & Validation

---

| Test / Module        | Expected Result                                  | Observed Result                                      | Status |
|----------------------|--------------------------------------------------|------------------------------------------------------|--------|
| AVL Spatial Indexing | Debris cleanly grouped by unique 3D sector key.  | Search returns only objects within exact local area. | Pass   |
| Min-Heap Sorting     | Debris with shortest distance is targeted first. | Successfully isolates nearest threat.                | Pass   |
| Input Validation     | Program should not crash if user types a letter. | Safely rejects string inputs and asks again.         | Pass   |
| Telemetry Export     | Write X, Y, Z data at every tick to a file.      | telemetry.csv is successfully generated.             | Pass   |

# GitHub Repository & Development Activity

---

## Repository Information

- ▶ **GitHub Link:** <https://github.com/Chhavit09/SHMCA>
- ▶ **Branch:** main
- ▶ **Contributors:** All Team Members

## Development Activity

- ▶ Separating files completely solved our merge conflicts.
- ▶ Code follows 3rd-semester syllabus requirements.

**Recent Commits:** 204e3a8 (Search Tool), d27d767 (Fuel Logic), 20bccdb (Input Checks), df655b9 (Nearest Sat Dist).

# Individual Contribution

---

| Member         | Role        | Technical Contribution                                                                         | GitHub Commits |
|----------------|-------------|------------------------------------------------------------------------------------------------|----------------|
| Satyam Kumar   | DSA Dev     | Implemented AVL_Tree.h, Vector math, and the simulated gravity physics for circular orbits.    | 8+ Commits     |
| Sudhanshu Ojha | DSA Dev     | Developed Queues_DSA.h (Min-Heap/Queues), distance math, and fixed the input buffer crashes.   | 5+ Commits     |
| Chhavit        | Team Lead   | Built Satellites_OOP.h, managed active fuel consumption, and added nearest satellite tracking. | 6+ Commits     |
| Harsh Jain     | DSA/Sim Dev | Constructed Chaos_Engine.h, FlightComputer.h, and the linear search tool for the main file.    | 6+ Commits     |

# Challenges, Remaining Work & Phase-III Roadmap

---

## Challenges We Overcame

- ▶ **Merge Conflicts:** Fixed by splitting into 5 modular files.
- ▶ **Input Crashes:** Handled using `cin.clear()` inside while loops.
- ▶ **Linear Physics:** Satellites initially flew away in straight lines until we added gravity math.

## Phase-III Roadmap

1. **Stop using the terminal.**
2. Design a visual frontend web application using HTML and CSS.
3. We will use JavaScript to read generated `telemetry.csv` file.
4. Visually animate the satellites and debris on the screen.

# Expected Outcomes & References

---

## Current Outcomes (Phase II)

- ▶ A completed C++ backend utilizing OOP and DSA concepts.
- ▶ Fully automated collision avoidance and fuel tracking.
- ▶ Successful generation of `blackbox.txt` and `telemetry.csv` files.

## References

1. B.Tech 3rd-Sem C++ Syllabus (OOP/DSA).
2. Algorithms for self-balancing AVL Trees.
3. Basic 3D math for coordinate distances.
4. C++ documentation for Inheritance/Polymorphism.
5. Priority Queue designs using Min-Heaps.

**Thank You**  
Questions & Suggestions